

Inheritance

- A class can inherit from other class
- It improves code reuse
- Constructor, attributes and methods are inherited to child class
- Parent class has no access to the child class
- child class methods overrides the parent class methods
 - this is called method overriding
- `super()` method is used to invoke parent class methods and constructor

Single inheritance

```
In [9]: 1 class Person:
        2     def __init__(self, name):
        3         self.name = name
        4     def show(self):
        5         print(self.name + " is a python teacher")
        6 class Professor(Person):
        7     def display(self):
        8         print(f"This is {self.name}")
```

```
In [6]: 1 per = Professor("Guido Rossum")
```

```
In [7]: 1 per.name
```

```
Out[7]: 'Guido Rossum'
```

```
In [8]: 1 per.show()
```

Guido Rossum is a python teacher

```
In [10]: 1 per1 = Person("James")
```

```
In [11]: 1 per1.name
```

```
Out[11]: 'James'
```

```
In [12]: 1 per1.show()
```

```
James is a python teacher
```

Multilevel inheritance

```
In [13]: 1 class Human:
2         def __init__(self,name):
3             self.name = name
4         def show(self):
5             print(self.name)
6         class Person(Human):
7             pass
8         class Professor(Person):
9             pass
10
```

```
In [14]: 1 p = Professor("Guido Rossum")
```

```
In [15]: 1 p.name
```

```
Out[15]: 'Guido Rossum'
```

```
In [16]: 1 p.show()
```

```
Guido Rossum
```

Multiple Inheritance

```
In [17]: 1 class Person:
2         def __init__(self,name):
3             self.name = name
4         def show(self):
5             print(self.name)
6 class Teacher:
7     def credit(self):
8         print("He teaches python")
9 class PythonProgrammer(Person,Teacher):
10     pass
```

```
In [18]: 1 p1 = PythonProgrammer("Guido Rossum")
```

```
In [19]: 1 p1.credit()
```

He teaches python

```
In [20]: 1 p1.name
```

```
Out[20]: 'Guido Rossum'
```

```
In [21]: 1 p1.show()
```

Guido Rossum

Hiearchical Inheritance

```
In [22]: 1 class Human:
2         def __init__(self,name):
3             self.name = name
4         def show(self):
5             print(self.name + " is a Professor")
6 class PythonProgrammer(Human):
7     pass
8 class JavaProgrammer(Human):
9     pass
```

```
In [23]: 1 p = PythonProgrammer("Guido")
```

```
In [24]: 1 p.show()
```

Guido is a Professor

```
In [25]: 1 j = JavaProgrammer("James Gosling")
```

```
In [26]: 1 j.show()
```

James Gosling is a Professor

Constructor Overriding

```
In [27]: 1 class Father:
2         def __init__(self):
3             self.money = 10000
4             print("Father class constructor")
5         def show(self):
6             print("Father class method")
7     class Son(Father):
8         def __init__(self):
9             self.money = 500
10            self.car = "BMW"
11            print("Son class constructor")
12        def display(self):
13            print("Son class method")
```

```
In [28]: 1 s = Son()
```

Son class constructor

```
In [29]: 1 s.money
```

Out[29]: 500

```
In [30]: 1 f = Father()
```

Father class constructor

```
In [31]: 1 f.money
```

Out[31]: 10000

super() method

- If we write constructor in both parent and child class, the parent class constructor is not available in child class
- `super()` method is used to call parent class from the child class

```
In [32]: 1 class Phone:
2         def __init__(self,price,brand,camera):
3             print("Inside phone class")
4             self.price = price
5             self.brand = brand
6             self.camera = camera
7         def buy(self):
8             print("Buying a phone")
9     class Smartphone(Phone):
10        def buy(self):
11            print("Buying a smartphone")
12            super().buy()
13
```

```
In [33]: 1 s = Smartphone(20000,"Iphone",13)
```

Inside phone class

```
In [34]: 1 s.buy()
```

Buying a smartphone
Buying a phone

another example

```
In [35]: 1 class Father:
2         def __init__(self):
3             self.money = 10000
4             print("Father class constructor")
5         def show(self):
6             print("Father class method")
7     class Son(Father):
8         def __init__(self):
9             super().__init__()
10            print("Son class Constructor")
11        def display(self):
12            print("Son class method")
13
```

```
In [36]: 1 s = Son()
```

```
Father class constructor
Son class Constructor
```

```
In [37]: 1 s.money
```

```
Out[37]: 10000
```

Polymorphism

- Condition of occurrence in different forms
- At operator level
- At function level
- At method level

Example

```
In [38]: 1 class Car:
2         def start(self):
3             print("Engine started")
4         def move(self):
5             print("Car is running")
6         def stop(self):
7             print("Brakes applied")
8     class Clock:
9         def move(self):
10            print("Tick Tick Tick")
11
12        def stop(self):
13            print("Clock stopped")
14    class Person:
15        def move(self):
16            print("Person is moving")
17        def stop(self):
18            print("Person stopped")
```

Method Overloading

```
In [39]: 1 c = Car()
2         cl = Clock()
3         p = Person()
```

```
In [40]: 1 def dosomething(x):
2         x.move()
3         x.stop()
```

```
In [41]: 1 dosomething(c)
```

Car is running
Brakes applied


```
In [42]: 1 dosomething(c1)
```

```
Tick Tick Tick  
Clock stopped
```

```
In [43]: 1 dosomething(p)
```

```
Person is moving  
Person stopped
```

Operator overloading

```
In [44]: 1 2 + 3
```

```
Out[44]: 5
```

```
In [45]: 1 "2" + "3"
```

```
Out[45]: '23'
```

Function overloading

```
In [46]: 1 print(len('python'))  
2 print(len(["ver1", "ver2", "ver3"]))  
3 print(len({"Creator": "Guido", "Nationality": "Dutch"}))
```

```
6  
3  
2
```

ABSTRACT CLASS

- A class derived from ABC class which belongs to abc module is known as abstract class in python
- ABC class is known as meta class which means class that defines behaviour of other classes.

- Abstract class can have concrete and abstract methods
- Abstract class needs to be extended and its methods implemented
- One cannot create objects of an abstract class

```
In [47]: 1 from abc import ABC, abstractmethod
2
3 class DefenceForce(ABC):
4
5     def __init__(self):
6         self.id = 101
7
8     @abstractmethod
9     def area(self):
10         pass
11     def gun(self):
12         print("The Gun is AK 47")
13
14
```

```
In [52]: 1 class Army(DefenceForce):
2         def area(self):
3             print("Area is land", self.id)
```

```
In [53]: 1 a = Army()
```

```
In [54]: 1 a.area()
```

Area is land 101

```
In [56]: 1 a.gun()
```

The Gun is AK 47

```
In [57]: 1 a.id
```

Out[57]: 101

```
In [58]: 1 class Navy(DefenceForce):
          2     def area(self):
          3         print("Area is Sea", self.id)
```

```
In [59]: 1 n = Navy()
```

```
In [61]: 1 n.area()
```

Area is Sea 101

```
In [63]: 1 n.gun()
```

The Gun is AK 47

Create an abstract class Shape with abstract method `calculate_area`. Create two class Rectangle and circle inheriting from Shape. Create two objects of rectangle and circle and find the area of these objects

632

Implement the following hierarchy . The Book class has name, n (number of authors), authors (list of authors), publisher, ISBN, and year as its data members and the derived class has course as its data member. The derived class method overrides (extends) the methods of the base class.

```
In [ ]: 1 class Book:
2     def __init__(self, name, n, authors, publisher, ISBN, year):
3         self.name = name
4         self.n = n
5         self.authors = authors
6         self.publisher = publisher
7         self.ISBN = ISBN
8         self.year = year
9     def display(self):
10        print(self.name)
11        print(self.n)
12        print(self.authors)
13        print(self.publisher)
14        print(self.ISBN)
15        print(self.year)
16 class DerivedBook(Book):
17     def __init__(self, name, n, authors, publisher, ISBN, year, course):
18         #super().__init__()
19         self.course = course
20     def display(self):
21         #super().display()
22         print(self.name)
23         print(self.n)
24         print(self.authors)
25         print(self.publisher)
26         print(self.ISBN)
27         print(self.year)
28
29
```

METHOD RESOLUTION ORDER IN PYTHON

- It defines an order in which the class is searched in case of multiple inheritance.
- It is based on C3 Linearization Algorithm ($\geq$ Python 2.3)

C3 Linearization Algorithm

Notation

- For a list of classes - C1C2C3..Cn
 - the C1 is the head of class and the rest is tail
- C3 Linearization Algorithm is defined as MRO of sum of any class C and the linear of parents and the list of parents

$$L(C(B1,B2\dots BN)) = C + \text{merge}(L(B1), L(B2) \dots BN))$$

- Here B1 to BN are the parent classes of C
1. Take the head of first list L(B1) from left to right
 2. If the head of selected class is not in the tail of the list, add it to the linear and remove it from the list
 3. Repeat the process until the classes are removed or it is impossible to find good head

In [1]:

```
1 !python --version
```

Python 3.8.5

EXAMPLE DEMONSTRATION

```
In [3]: 1 class F:
        2     pass
        3 class E:
        4     pass
        5 class D:
        6     pass
        7 class C(D,F):
        8     pass
        9 class B(D,E):
       10     pass
       11 class A(B,C):
       12     pass
```

```
In [4]: 1 A.mro()
```

```
Out[4]: [__main__.A,
         __main__.B,
         __main__.C,
         __main__.D,
         __main__.E,
         __main__.F,
         object]
```

First step

- MRO OF D,E,F,O
- L(D) = DO
- L(E) = EO
- L(F) = FO
- L(O) = O

Second step

$$\begin{aligned}L(C) &= C + \text{MERGE}(L(D), L(F), DF) \\&= C + \text{MERGE}(DO, FO, DF) \\&= C + D + \text{MERGE}(O, FO, F) \\&= C + D + F + O \\&= CDFO\end{aligned}$$

Third Step

$$\begin{aligned}L(B) &= B + \text{MERGE}(L(D), L(E), DE) \\&= B + \text{MERGE}(DO, EO, DE) \\&= B + D + \text{MERGE}(O, EO, E) \\&= B + D + E + O \\&= BDEO\end{aligned}$$

Fourth step

$$\begin{aligned}L(A) &= A + \text{MERGE}(L(B), L(C), BC) \\&= A + \text{MERGE}(BDEO, CDFO, BC) \\&= A + B + \text{MERGE}(DEO, CDFO, C) \\&= A + B + C + \text{MERGE}(DEO, DFO) \\&= A + B + C + D + \text{MERGE}(EO, FO) \\&= A + B + C + D + E + F + O \\&= ABCDEFO\end{aligned}$$

PB 604

```
In [5]: 1 class A:           # A0
        2     pass
        3 class B:           # B0
        4     pass
        5 class C:           # C0
        6     pass
        7 class X(A,B):       # XAB0
        8     pass
        9 class Y(C,A,B):     # YCAB0
       10     pass
       11 class Z(A):         # ZAO
       12     pass
       13 class P(Z,Y,X):      # P + MERGE(ZAO, YCAB0, XAB0, ZYX)
       14     pass
```

```
In [6]: 1 P.mro()
```

```
Out[6]: [__main__.P,
         __main__.Z,
         __main__.Y,
         __main__.C,
         __main__.X,
         __main__.A,
         __main__.B,
         object]
```


Inconsistent MRO Design

In [7]:

```
1 class A:
2     pass
3 class B:
4     pass
5 class C(A,B):
6     pass
7 class D(B,A):
8     pass
9 class X(C,D):
10    pass
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-7-e6066b25c528> in <module>
      7 class D(B,A):
      8     pass
----> 9 class X(C,D):
     10     pass
```

TypeError: Cannot create a consistent method resolution
order (MRO) for bases A, B

OPERATOR OVERLOADING

```
__add__ = +
__sub__ = -
__mul__ = *
__truediv__ = /
```

```
In [8]: 1 class Fraction:
2
3     def __init__(self,x,y):
4         self.num = x
5         self.den = y
6
7     def __add__(self,other):
8         new_num = self.num*other.den + other.num*self.den
9         new_den = self.den * other.den
10        return f"{new_num}/{new_den}"
11
12    def __sub__(self,other):
13        new_num = self.num*other.den - other.num*self.den
14        new_den = self.den * other.den
15        return f"{new_num}/{new_den}"
16
17    def __mul__(self,other):
18        new_num = self.num * other.num
19        new_den = self.den * other.den
20        return f"{new_num}/{new_den}"
21
22    def __truediv__(self,other):
23        new_num = self.num * other.den
24        new_den = self.den * other.num
25        return f"{new_num}/{new_den}"
26
27    def __str__(self):
28        return f"{self.num} / {self.den}"
29
30    def convert_decimal(self):
31        return self.num / self.den
32
33
```

```
In [ ]: 1 Fraction1 = 1/3
2        Fraction2 = 2/5
```

```
In [9]: 1 f1 = Fraction(1,3)
        2 f2 = Fraction(2,5)
```

```
In [10]: 1 f1 + f2
```

```
Out[10]: '11/15'
```

```
In [11]: 1 f1*f2
```

```
Out[11]: '2/15'
```

```
In [12]: 1 f1 / f2
```

```
Out[12]: '5/6'
```

```
In [13]: 1 print(f1)
        2 print(f1.convert_decimal())
```

```
1 / 3
0.3333333333333333
```

Comparison Operators

```
__eq__ -> ==
__lt__ -> <
__gt__ -> >
```

WAP to create a class Data with data member value. Create two objects of the class to demonstrate comparison operators

PB 634

Create a class called Student, having name and email as its data members and `_init_(self, name, email)` and `putdata(self)` as bound methods. The `_init_` function should assign the values passed as parameters to the requisite variables. The `putdata` function should display the data of the student. Create another class called PhDguide having name, email, and students as its data members. Here, the students variable is the list of students under the guide. The PhDguide class should have four bound methods: `_init_`, `putdata`, `add`, and `remove`. The `_init_` method should initialize the variables, the `putdata` should show the data of the guide, include the list of students, the `add` method should add a student to the list of students of the guide and the `remove` function should remove the student (if the student exists in the list of students of that guide) from the list of students.

```
In [ ]: 1 class Student:
2     def __init__(self,name,email):
3         self.name = name
4         self.email = email
5     def putdata(self):
6         print(self.name)
7         print(self.email)
8 class PhDGuide:
9     def __init__(self,name,email):
10        self.name = name
11        self.email = email
12        self.students = []
13
14    def put_data(self):
15        print("Guide name", self.name)
16        print("Guide email", self.email)
17        print("Students under guide")
18
19        for student in self.students:
20            student.putdata()
21            print("-----")
22    def add(self,student):
23        if student not in self.students:
24            self.students.append(student)
25            print("Student added")
26    def remove(self,student):
27        if student in self.students:
28            self.students.remove(student)
29            print("Student removed")
30        else:
31            print("Student not found")
32
33
34
```

Numpy

```
In [1]: 1 import numpy as np
```

```
In [5]: 1 arr = np.array([1,2,3,4,5])
```

```
In [7]: 1 print(arr)
2 print(type(arr))
3 print(np.__version__)
```

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
1.19.2
```

2D array

```
In [8]: 1 b = np.array([[1,2,3],[4,5,6]])
2 print(b)
```

```
[[1 2 3]
 [4 5 6]]
```

3D array

```
In [12]: 1 c = np.array([[[1,2],[3,4]],[[5,6],[7,8]]],dtype = float)
```

```
In [13]: 1 print(c)
```

```
[[[1. 2.]
  [3. 4.]]

 [[5. 6.]
  [7. 8.]]]
```

dtype

```
In [14]: 1 c.dtype
```

```
Out[14]: dtype('float64')
```

```
In [15]: 1 arr = np.array([1,2,3,0,0], dtype = bool)
```

```
In [16]: 1 print(arr)
```

```
[ True  True  True False False]
```

Array attributes

ndim - return number of dimensions

```
In [29]: 1 a = np.array([0,1,2,3,4,5,6,7,8,9])  
        2 b = np.array([[1,2,3],[4,5,6]])  
        3 c = np.array([[[1,2],[3,4]], [[5,6],[7,8]]], dtype = float)
```

```
In [21]: 1 a2 = np.arange(10)
```

```
In [30]: 1 print(a)
          2 print(b)
          3 print(c)
```

```
[0 1 2 3 4 5 6 7 8 9]
[[1 2 3]
 [4 5 6]]
[[[1. 2.]
   [3. 4.]]

 [[5. 6.]
  [7. 8.]]]
```

```
In [31]: 1 print(a.ndim)
          2 print(b.ndim)
          3 print(c.ndim)
```

```
1
2
3
```

shape - to check the shape

```
In [32]: 1 print(a.shape)
          2 print(b.shape)
          3 print(c.shape)
```

```
(10,)
(2, 3)
(2, 2, 2)
```


size of the array

```
In [33]: 1 print(a.size)
          2 print(b.size)
          3 print(c.size)
```

```
10
6
8
```

itemsize - size in bytes of an array item

```
In [34]: 1 print(a.itemsize)
          2 print(b.itemsize)
          3 print(c.itemsize)
```

```
4
4
8
```

dtype

```
In [35]: 1 print(a.dtype)
          2 print(b.dtype)
          3 print(c.dtype)
```

```
int32
int32
float64
```

Reshape

```
In [39]: 1 a1 = np.arange(12).reshape(2,6)
```

```
In [40]: 1 print(a1)
```

```
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

Scalar operation

```
In [43]: 1 print(a1 + 2)
         2 print(a1) # VIEW ONLY
```

```
[[ 2  3  4  5  6  7]
 [ 8  9 10 11 12 13]]
[[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]
```

```
In [44]: 1 print(a1*2)
         2 print(a1/2)
         3 print(a1//2)
         4 print(a1**2)
```

```
[[ 0  2  4  6  8 10]
 [12 14 16 18 20 22]]
[[0.  0.5 1.  1.5 2.  2.5]
 [3.  3.5 4.  4.5 5.  5.5]]
[[0 0 1 1 2 2]
 [3 3 4 4 5 5]]
[[ 0  1  4  9 16 25]
 [36 49 64 81 100 121]]
```

np.sort - view only

- Returns a sorted copy of an array

```
In [45]: 1 a = np.array([58,67,12,45,53,89,93])
```

```
In [48]: 1 np.sort(a)[::-1]
```

```
Out[48]: array([93, 89, 67, 58, 53, 45, 12])
```

```
In [47]: 1 a
```

```
Out[47]: array([58, 67, 12, 45, 53, 89, 93])
```

np.concatenate

- It concatenates a sequence of arrays along the existing axis

```
In [56]: 1 c = np.arange(6).reshape(2,3)
2 d = np.arange(6,12).reshape(2,3)
3 print(c)
4 print(d)
```

```
[[0 1 2]
 [3 4 5]]
[[ 6  7  8]
 [ 9 10 11]]
```

```
In [58]: 1 np.concatenate((c,d),axis=1)
```

```
Out[58]: array([[ 0,  1,  2,  6,  7,  8],
                [ 3,  4,  5,  9, 10, 11]])
```

```
In [59]: 1 np.concatenate((c,d),axis=0)
```

```
Out[59]: array([[ 0,  1,  2],
                [ 3,  4,  5],
                [ 6,  7,  8],
                [ 9, 10, 11]])
```

```
In [ ]: 1
```

```
In [ ]: 1
```

Copy in list vs numpy

```
In [49]: 1 a = [1,2,3]
          2 b = a
          3 b[1] = 100
          4 print(a)
```

```
[1, 100, 3]
```

np.nditer

```
In [50]: 1 a = np.array([58,67,12,45,53,89,93])
```

```
In [51]: 1 for i in np.nditer(a):
          2     print(i)
```

```
58
67
12
45
53
89
93
```

```
In [52]: 1 a = np.array([0,1,2,3,4,5,6,7,8,9])
          2 b = np.array([[1,2,3],[4,5,6]])
          3 c = np.array([[[1,2],[3,4]],[[5,6],[7,8]]],dtype = float)
```

```
In [53]: 1 for i in b:
          2     print(i)
```

```
[1 2 3]
[4 5 6]
```

```
In [55]: 1 for i in np.nditer(b):
          2     print(i, end = " ")
```

```
1 2 3 4 5 6
```

np.where

- It returns the indices of elements in an input array where the given condition is satisfied

```
In [60]: 1 a = np.array([56,68,23,11,78,86,34,51,50])
```

```
In [61]: 1 np.where(a>50)
```

```
Out[61]: (array([0, 1, 4, 5, 7], dtype=int64),)
```

```
In [62]: 1 np.where(a>50,0 ,a)
```

```
Out[62]: array([ 0,  0, 23, 11,  0,  0, 34,  0, 50])
```

```
In [63]: 1 a
```

```
Out[63]: array([56, 68, 23, 11, 78, 86, 34, 51, 50])
```

```
In [64]: 1 np.where(a%2==0,0 ,a)
```

```
Out[64]: array([ 0,  0, 23, 11,  0,  0,  0, 51,  0])
```

Splitting Arrays

- It is reverse of joining
- It breaks array into multiple arrays
- `array_split` is used with number of split as parameters to split the array

```
In [65]: 1 import numpy as np
```

```
In [70]: 1 arr = np.array([1,2,3,4,5,6,7])
```

```
In [72]: 1 np.array_split(arr,2)
```

```
Out[72]: [array([1, 2, 3, 4]), array([5, 6, 7])]
```

```
In [73]: 1 np.array_split??
```

random in numpy

randint

```
In [78]: 1 from numpy import random  
2 x = random.randint(100)  
3 print(x)
```

rand

In [81]:

```
1 from numpy import random
2 x = random.rand(100)
3 print(x)
```

```
[0.44490552 0.64618986 0.57557964 0.81693814 0.39966064 0.59020909
 0.50163663 0.99203517 0.39463515 0.05384282 0.96382311 0.71170475
 0.48497938 0.95877866 0.77707155 0.13424292 0.40243714 0.60397887
 0.60482901 0.6739818 0.31135235 0.73939564 0.55729475 0.44002373
 0.70240003 0.43854956 0.5497212 0.08876453 0.71540537 0.91635155
 0.9079892 0.58984107 0.76259223 0.05716972 0.31294993 0.08924517
 0.13622525 0.2358049 0.34772046 0.53683034 0.73105105 0.26828679
 0.56217566 0.77444847 0.60790882 0.35645604 0.5449872 0.0346265
 0.6020992 0.99641143 0.49700611 0.87340058 0.78073184 0.82914681
 0.8808458 0.28804273 0.83837798 0.3560302 0.7987659 0.83413464
 0.27678907 0.32641032 0.81388265 0.15070895 0.13498193 0.75332995
 0.92791003 0.88316601 0.62565036 0.50797811 0.40259678 0.98654346
 0.99017491 0.301188 0.24001948 0.92158028 0.22388519 0.99779452
 0.0638552 0.79739631 0.70845863 0.79405097 0.16990709 0.26053732
 0.29105486 0.53696279 0.40010931 0.42471525 0.54982623 0.49801738
 0.92320736 0.59021823 0.10763819 0.96051727 0.40688697 0.72484373
 0.23357249 0.5047406 0.15919169 0.79460123]
```

In [80]:

```
1 from numpy import random
2 x = random.rand()
3 print(x)
```

```
0.049413251470032615
```

1 d array

```
In [84]: 1 from numpy import random
          2 x = random.randint(100, size=(5))
          3 print(x)
```

```
[43 89 28 94 80]
```

2d array

```
In [85]: 1 from numpy import random
          2 x = random.randint(100, size=(3,5))
          3 print(x)
```

```
[[ 3 66 95 16 15]
 [62 43 82 83  2]
 [54 96 69 89 54]]
```

```
In [88]: 1 from numpy import random
          2 x = random.rand(5)
          3 print(x)
```

```
[0.63565914 0.93538102 0.65906207 0.48681843 0.95139038]
```

```
In [89]: 1 from numpy import random
          2 x = random.rand(3,5)
          3 print(x)
```

```
[[0.72581867 0.21695288 0.30481771 0.36026306 0.53440294]
 [0.21957001 0.93445532 0.14591345 0.00755266 0.39877984]
 [0.28525916 0.8964515  0.176928   0.6170826  0.48167336]]
```

choice method


```
In [96]: 1 x = random.choice([3,5,6,7,8])
```

```
In [97]: 1 print(x)
```

3

```
In [99]: 1 from numpy import random  
2 x = random.choice([3,5,7,9], size=(3,5))  
3 print(x)
```

```
[[5 5 3 9 7]  
 [7 3 5 5 7]  
 [9 5 5 9 9]]
```